PREPRINT • TECHNICAL REPORT · OCTOBER 2026

CRAP: Compiler Replacement for Assembling Programs

Fine-tuning a small language model to emit raw ELF executables under an execution-based verifiable reward

Matthew Haynes[1]

[1] Independent Researcher

DATE October 26, 2026 CODE matthewhaynesonline/crap-public

MODELS hf.co/collections/matthewhaynesonline/crap11 BLOG blog.studiohaynes.com/go/crap

ABSTRACT

CRAP is a small language model fine-tuned for generating executables directly. Given a natural-language request for a program, it emits the finished ELF executable as raw bytes (encoded as hex), with no compiler, assembler or linker involved. We trained a LoRA adapter (a small set of extra weights on top of the frozen base model) for Gemma-4-E2B-it by supervised fine-tuning on a templated dataset. Every output is graded by executing it, which we call an *executable verifiable reward*: each output is parsed from hex, structurally validated, executed in a sandbox and checked for the requested behavior. The released model passes 87.0% of the legacy held-out behavioral evaluation. It also handles novel strings, wider exit codes and multi-turn requests; success rates on those axes are reported separately from the 87.0% figure. It learns the ELF structure and boilerplate early in training, but struggles with operand binding (putting the exact value in the right field). The system is meant as a minimal, fully verifiable test bed for fine-tuning grounded in execution. During training, conventional signals repeatedly diverged from real performance: token accuracy decoupled from task success, and sequence packing silently mixed samples across document boundaries. At serving time, a thinking token and default sampler penalties broke byte-exact output. Three training runs on three

corpora show robustness tracking the corpus (though the runs do not separate corpus composition from training budget or dataset size). With a trained-code control, they also give the project's sharpest case of a misleading proxy metric: one run reached 0.038 loss, with every exit-code binary structurally valid, while generalizing to almost none of the held-out exit codes.

KEYWORDS binary generation · verifiable rewards · execution-grounded fine-tuning · ELF

CONTENTS

1 Introduction 2
2 Background 4
3 The CRAP System 5
3.3 The Evaluation Harness 5
4 Dataset Construction and Iteration 7
5 Training Infrastructure 8
6 Serving Fidelity 9
7 Results 11
8 Failure Analysis 13
9 Discussion 15
10 Related Work 16
11 Conclusion 17

1 Introduction

Today's programs[1] go through a toolchain before they run: source is compiled, assembled and linked into an executable by deterministic machinery. This holds whether a human or a language model wrote the source. This report asks whether a language model can skip that process entirely and emit the final bytes of a runnable executable, header to syscalls, with no compiler, assembler or linker in the loop, and whether that output can be checked rigorously rather than by proxy.

Let us examine a hello world program through various layers to determine

1. "Program" here means a compiled program; an interpreted one merely delegates the problem to an interpreter that was itself compiled.

how much of the toolchain falls away. A default Rust production build is roughly 450 KB; a C build is roughly 16 KB, as is compiler-generated assembly; assembly that makes raw syscalls gets down to about 9 KB; and a stripped-down minimal binary lands under 200 bytes (source for each rung is in Appendix A). Each step removes another part of the toolchain until only the bytes the kernel requires are left. So the natural target is a model that emits those bytes directly.[2]

Binaries make a useful test bed for one reason: they are maximally unforgiving. One wrong byte can segfault. ELF structure is rigid and machine-checkable, and behavior can be verified by running the binary. That kind of precision is rare in language model evaluation, and it gives a deterministic, verifiable reward signal with no human judge or separate reward model required.

The contributions are:

1. **A demonstration** that a fine-tuned small language model (Gemma-4-E2B-it with a 24.2M-parameter LoRA adapter) can directly emit working Linux executables, passing 87.0% of the legacy held-out behavioral evaluation[3] with generalization reported separately by axis.

2. **The CRAP system**: a programmatic binary-generation library, a templated dataset generator and an evaluation harness that runs every output, whose score can be used directly as a verifiable reward.

3. **A dataset debugging case study**: three dataset bugs where loss and token accuracy looked fine while the model failed the actual task.

4. **A corpus composition progression** across three training runs, read as an evolution rather than a controlled experiment: robustness tracks the corpus, but composition, training budget and dataset size all changed together. A *trained-code control* (scoring a run on the exact exit codes it trained on) shows the intermediate run failing even on exit values it had seen. The released model's held-out successes are generalization within this family of tasks, not proof that it learned a general conversion algorithm. The same control gives the project's sharpest case of a misleading proxy metric (Section 9).

5. **A training infrastructure finding**: in our `SFTTrainer` configuration, sequence packing without boundary masking let packed samples attend across document boundaries, silently contaminating training (training examples could see each other). FlashAttention's variable-

2. Elon Musk has predicted that by the end of 2026 "the AI just creates the binary directly" [15]. This model does so, slightly ahead of schedule, for programs that print a string and exit.

3. That is, 87% of the time, it works every time. Decoding is greedy, so a prompt that passes once passes every time, as long as the serving setup stays pinned (precision, kernels and the prompt template can still flip a borderline output).

length kernels provide that masking but would not build on our stack.

6. **A serving case study**: a thinking token in the chat template and the default sampler penalties broke byte-exact output; a token-level prompt differ found the mismatch and verified that serving matched training again.

2 Background

2.1 Anatomy of a minimal executable

A minimal Linux x86-64 executable consists of an ELF header, one program header describing a single loadable segment and a short instruction sequence that calls the `write` and `exit` syscalls, followed by any string data. Figure 1 annotates the full byte layout of such an executable.



Figure 1 Anatomy of a minimal ELF executable (the 193-byte `hello_minimal` of Appendix A). Every internal reference agrees with every other: `e_entry` points at the first code byte, the string-load displacement points at the payload, and `p_filesz` equals the file length. Maintaining this agreement is the job of the compiler and linker; here the model must produce it directly.

2.2 The bytes are the artifact

The model's output is the final executable, not source code or some other intermediate representation. Hex is only the transport encoding (two hex characters per byte), so "emitting the program" and "emitting the hex" are the same thing. That means the model's output needs the same internal consistency a compiler and linker would guarantee (Figure 1).

2.3 Platform asymmetry

Linux ELF static binaries can be trimmed to the bare minimum: a working print-and-exit binary is about 190 bytes. macOS Mach-O cannot: the AMFI subsystem requires code signing, segments must respect 16 KB page alignment and every executable has to go through the system loader (`dyld`) and link `libSystem`. So the theoretical floor for a minimal Mach-

O binary is about 16,484 bytes (the working Mach-O binaries CRAP builds run 33 to 51 KB). For this reason, while the broader CRAP system supports Mach-O, fine-tuning and training only target Linux x86-64 ELF.

2.4 Why direct byte generation is hard

Generating a deterministic program with a probabilistic model runs into three problems. First, bytes and tokens do not align: the base model's tokenizer was not designed for hex, so a single meaningful unit (a field, an instruction) spans an unpredictable number of tokens. Second, *operand binding*: the right value has to land in the right field. An exit code must be the immediate of the right instruction, not merely *somewhere* in the output. Third, *length counting*: the `write` syscall takes an explicit byte count that must match the string, so the model must produce the hex for the string and also the hex for that string's length.

3 The CRAP System

3.1 The binary-gen library

The `binary-gen` library builds binaries field by field (struct-based[4]) and is the ground truth for training and evaluation. It supports `print`, `exit` and `print_exit` actions on Linux (x86-64 ELF) and macOS (Arm64 Mach-O). An `analysis` module built on `lief` parses and validates arbitrary binaries; it is used both to verify the dataset and in the evaluation harness.

4. Struct as in Python's `struct` packer: every field is packed to bytes explicitly, offset by offset.

3.2 Dataset generation

Training examples pair a natural-language request ("print ahoy matey and exit with code 3") with the verified bytes `binary-gen` produces for it. The dataset generator takes every combination of platform × sample string × exit code and varies the system prompt: about 60% of examples use CRAP's default system prompt, 20% use variants and 20% use none.[5] It runs from a CLI, and generation is deterministic under a fixed seed.

5. The variation is meant to keep the model from depending on one exact system prompt. Evaluation uses only the default prompt, so this report does not measure that robustness.

3.3 The evaluation harness as a verifiable reward

The evaluation harness is the heart of the system. Every model output goes through three stages:

1. **Hex parse**: first and foremost, the output has to decode to valid bytes.
2. **Structure validation**: the bytes must parse as a well-formed executable for the intended platform (checked by the `analysis` module: headers, segments and entry point).

3. **Behavioral validation**: the binary runs (in a Docker sandbox by default), and its stdout and exit code are compared with the request.

An output passes only if it clears all three; there is no partial credit from the kernel.

DEFINITION 1 · EXECUTABLE VERIFIABLE REWARD

A deterministic score computed by running the model's output. No reward model, no rubric, no human judgment: it can serve directly as the reward function for GRPO-style reinforcement learning with verifiable rewards (training where the reward comes from an automatic, objective check) [12, 7].

The idea comes from the execution-based evaluation of code models [1]: parse, run, check; no proxy metrics needed.

Two caveats apply. First, the sandbox only runs Linux x86-64. Mach-O outputs can be built and validated by `binary-gen`, but Mach-O is out of scope for the size and complexity reasons stated above, so training and behavioral evaluation are Linux only. Second, the harness ignores newlines but not NUL bytes: a write length that is too long (emitting `ahoy\n\x00`) still fails, while one that is too short and only drops the trailing newline passes, since the newline is treated as cosmetic.[6]

The evaluation set is frozen and versioned (`EVAL_SET_VERSION = 2`; Appendix C). It starts from the original random holdout under fixed seed 99: 8 strings and 15 exit codes, 23 examples in all. This *legacy* split is the anchor for comparing runs. On top of it are 25 wider exit codes, 12 trained-code controls, 40 novel out-of-distribution strings, 8 combined print-and-exit requests, 12 multi-turn conversations and 24 doubled-character strings in 12 matched pairs. The multi-turn subset (four two-turn and eight three-turn cases) feeds the model a correct reference history and scores only its final reply, so it measures how much context the model can carry, not whether a conversation survives the model's own earlier mistakes.

Two more caveats apply when reading the results. First, the held-out exit-code metric (40 codes) is the legacy 15 plus the 25 wider codes, so it overlaps the legacy number rather than adding independent evidence. Second, it is *asymmetric across models*: the 25 wider codes were held out of the released model's training but were in the baseline's. The same score is therefore generalization for one model and recall for the other, so the two subsets are reported separately and the metric is never aggregated across models.

6. A strict structural assertion (write length equals payload length) would catch both directions but would re-fail the benign undercount; if added, it should flag only over-reads.

3.4 Training setup

The base model is Gemma-4-E2B-it [8], fine-tuned with `SFTTrainer` and a LoRA adapter [10] of 24.2M trainable parameters. Gemma 4 supports system prompts natively, hence the system prompt variation above. The fine-tuned model is used text only; multimodal and thinking features are out of scope (see Section 6). The eval harness runs during training, and the best checkpoint is picked by its evaluation result, not its loss.

4 Dataset Construction and Iteration

The first dataset iteration[7] had 184 examples covering `print` and `exit`. After 300 training steps the final logged loss was 0.015 with 99.4% token accuracy, yet the model passed only 42.9% (3 of 7) on the evaluation harness. Prints passed; exit codes failed.

Issue 1: data sampling. The exit-code training data came from a range that skipped values (`range(0, 256, 3)`), so the model had never seen some exit codes, including several in the evaluation set. This was widened to the full `range(0, 256)` on 2026-06-07, a week before the baseline run: the baseline and the negative specimen (Section 7) both trained on all 241 non-held-out codes. The sparse range is corpus history, not a property of any reported artifact.

Issue 2: composition gap. "Print sup and exit 3" produced a binary that printed and then exited 0. The initial training data had print samples *or* exit samples, never both, so the model had no reason to combine them. A combined `print_exit` action was added to `binary-gen` to fix that.

Issue 3: string diversity. "Hello world" came back as "hello worl". The pool of sample strings (`SAMPLE_STRINGS`) had too few natural multi-word strings, so it was expanded with more multi-word and long examples, plus digit and punctuation variants.

The corrected second iteration had 13× more examples, with full exit-code coverage, system prompt variation and the more diverse sample strings (Table 1).

Table 1 Dataset iterations.

7. These *dataset* iterations are numbered independently of the model versions (v1, v1.1) discussed from Section 7 onward, which name trained artifacts rather than corpora.

	Iteration 1	Iteration 2
Examples	184	2,475
Actions	`print, exit`	`print, exit, print_exit`
Strings	98 (2 natural multi-word)	127 (multi-word, long, punctuation)
Exit-code coverage	`range(0, 256, 3)`	0–255 pool (15 held out)
System-prompt variation	no	yes (∼60/20/20)

The second iteration exposed another trap. According to our notes, its first development training run scored 0% on the eval harness, seemingly far worse than the first, but it had covered only 0.13 epochs of the 13× larger dataset (its logs were not recovered). Encoding and structural (ELF format) failures cleared first and behavioral failures cleared much later.

5 Training Infrastructure

5.1 The "overnight" run that took 38.5 hours

The first full training run used Apple silicon (MPS): 1,000 steps, planned as an overnight job. It finished 38.5 hours later. The best checkpoint was at step 600, passing the eval harness at 82.6%; the final checkpoint fell back to 78.3%. The pass rate bounced around a plateau while loss kept falling, a pattern that will come up again.

5.2 Training inefficiently?

Profiling suggested that possibly 90% or more of compute was going to pad tokens (filler that makes every sample in a batch the same length): the training samples are short requests, not long multi-turn conversations, so most of each batch was padding. Sequence packing (concatenating several short samples into one sequence) looked like the obvious fix. It was not. Packing barely moved wall time on MPS, because it only fit about 5 examples per sequence, rather than the assumed ∼20. That buys more epochs for the same number of steps, but it does not meaningfully reduce the steps needed, or the time and cost that come with them.

5.3 Renting compute

After a first full run that took a day and a half on a laptop, the visionary "end of 2026" deadline looked at risk, so we rented an NVIDIA RTX 5090 for the later runs: for speed, to validate on a second platform and to stay closer to standard practice. The first run there, with packing on, looked fantastic: a loss of 0.04 by step 10. The evaluation pass rate disagreed, stalling around 20%. Without FlashAttention's variable-length (`cu_seqlens`) kernels [3],

packed samples attend across document boundaries, so every example in a packed sequence can see its neighbors. In this `SFTTrainer` setup, SDPA left document boundaries unmasked on MPS and CUDA alike. In other words, the low loss did not reflect task performance.[8]

5.4 Final training

We dropped sequence packing (`--no-packing`), and the next run finished 620 steps in 84 minutes. It peaked at 87.0% on the legacy split, first at checkpoint 300 and again at 450, 500 and 550 (Appendix B), beating the 38.5-hour MPS run in under 90 minutes. The cost of the first two CUDA runs was in line with cutting-edge AI research capex: about $2.50.

This run is the *baseline* model (v1) for the results that follow, with one caveat. Best-checkpoint loading was not wired into this run,[9] so its saved adapter is the *final* step-620 checkpoint (82.6% unmerged), not the 87.0% peak. The published, merged baseline scores 87.0%; the gap comes down to a single exit code, covered in Section 7.

Two more runs followed on the same configuration (packing off, SDPA, RTX 5090). Both went after the baseline's two biggest gaps: novel (out of distribution) strings and multi-turn requests, which the baseline had never trained on. The first (`jul19`) enriched the string data and added multi-turn examples, and lost nearly all of its exit-code ability in the process (Section 7). So the second, the *released* model (v1.1, `jul26`), added a third goal: keep the new robustness *and* recover exit-code coverage. It rebalanced the corpus to match, with multi-turn examples making up ~18% of it. Table 2 lists all five runs.

8. This is a known property of packing: with correct boundary masking, packing is benign [11]; the failure is in the masking, not the packing. FlashAttention-2 would not build on the Blackwell + cp314 stack, so masked packing was unavailable.

9. The evaluation-based checkpoint selection of Section 3.3 was added afterward; the two later campaign runs load their best checkpoint by evaluation result.

Table 2 Training runs across the campaign. **Peak** is the legacy 23-item behavioral pass rate at the run's best checkpoint. The MPS reference ran on Apple silicon; all CUDA runs used a rented RTX 5090.

Run	Config	Steps	Peak	Runtime
MPS ref	packing off, SDPA	1000	82.6% @ 600	38.5 h
Packing	packing on, SDPA	140	21.7% (stuck)	36 min
Baseline (v1)	packing off, SDPA	620	**87.0%** @ 300	84 min
Alpha (`jul19`)	packing off, SDPA	615	30.4% @ 450	76 min
Released (v1.1)	packing off, SDPA	1440	**87.0%** @ 800	~7.6 h

6 Serving Fidelity

Paradoxically, the best checkpoint from training did worse once served. Exported to GGUF f16 and served with llama.cpp, novel prompts produced

both badly encoded hex and correctly encoded but structurally invalid ELF binaries. The usual suspects were GGUF conversion and quantization rot. However, the culprit was neither.

At serving time, llama.cpp injected a single literal `<|think|>` token when rendering Gemma 4's chat template. The training data contained no think tokens, and thinking was not turned on. That one token was all it took to push a byte-exact model out of distribution. A token-level prompt differ, comparing the training-time prompt against the served prompt, found it; the other usual Gemma-plus-llama.cpp suspect, a doubled `<bos>` token, was ruled out.

The fix prevents think-token injection, pins greedy decoding and disables the repetition penalty:[10]

• serve.sh

```
# --reasoning off: no <|think|> injection
# --temp 0 --top-k 1: pin greedy
# --repeat-penalty 1.0: don't fight the 00 runs
llama-server -m crap-f16.gguf \
  --reasoning off \
  --temp 0 --top-k 1 \
  --repeat-penalty 1.0
```

LISTING 1 Serving flags for training/serving parity.

The repetition penalty needed the same treatment. It adjusts the model's output scores (logits) before sampling to discourage repetition, which is desirable in chat. Binaries, however, legitimately contain long runs of `00` bytes, and a penalty that fights them corrupts byte counts, padding and offsets. Greedy decoding does not neutralize it, since the penalty is applied before the greedy pick. With the fix, the served prompt is token-identical to training. If a build still injects the token, the fallback is to bypass templating entirely and feed a pre-tokenized prompt to llama.cpp's `/completion`.

We A/B diffed five trained strings from the Transformers reference implementation against the chat endpoint and the pre-tokenized `/completion`: all 5 were byte-identical on both, with no f16 drift. This is a deliberately limited smoke test; longer payloads and lower quants (such as Q4) were not tested.

With the serving fix in place, the served model recovers fully on single-turn requests. The baseline's f16 GGUF passes the legacy split at 20/23

10. Preventing token injection through the chat-template `enable_thinking` kwarg also works but is deprecated in current llama.cpp builds; `--reasoning off` is the recommended flag.

with zero structure-stage failures, identical to its merged `transformers` score. A later pass over the full frozen evaluation set (Section 7) found the released model's f16 GGUF identical to its merged bf16 weights on all but one out-of-distribution item, a single borderline greedy decode. That extends the five-string smoke test to the whole set. Multi-turn context is the one place a genuine structural collapse still appears (Section 8).

7 Results

Headline. The released model (v1.1) passes **87.0%** (20/23)[11] of the legacy held-out Linux behavioral evaluation. That ties the baseline (v1) on the split used to compare runs, and the released model improves on every generalization axis: novel strings, doubled-character strings, multi-turn requests and combined print-and-exit (Table 3). Measured against what the v1.1 training campaign set out to do, out-of-distribution strings go from 8 to 26 of 40, multi-turn from 0 to 10 of 12, and exit-code coverage holds. Each axis is reported on its own rather than blended into one number. One caveat up front: the legacy split also guided checkpoint selection and corpus development, so it is a validation set rather than an untouched final test.

11. With $n = 23$ the legacy anchor is coarse (one example is 4.3 percentage points), so mid-run swings of a few binaries should be read accordingly. The per-axis counts below are likewise small, so read them as directions, not tight estimates.

Table 3 Held-out results by axis, across the training progression (frozen evaluation set, best/merged artifact per run). **Legacy** is the 23-item cross-run anchor. **Exit** is the 40 held-out exit codes (it contains the legacy 15, so it is not independent of **Legacy**; the 25 wider codes are recall for v1 and generalization for v1.1). **Trained** is the trained-code control (12 codes each run trained on). **OOD** novel strings, **Dbl.** doubled-character strings (12 matched pairs), **Multi** multi-turn conversations, **P+E** combined print-and-exit.

Run	Legacy /23	Exit /40	Trained /12	OOD /40	Dbl. /24	Multi /12	P+E /8
v1 (baseline)	20	36	12	8	2	0	4
v1.1-alpha (`jul19`)	7	1	1	24	16	0	0
v1.1 (released)	**20**	**36**	**12**	**26**	**18**	**10**	**6**

Reading the runs in order. The intermediate run (v1.1-alpha) already had most of the string robustness gains: OOD 24/40 and doubles 16/24, close to the released model's 26 and 18. However, it lost nearly all of its exit-code ability (legacy 7/23, held-out exit 1/40). The released corpus was rebalanced to keep both, with multi-turn examples added on top (10/12, against 0/12 for both earlier runs). So string robustness tracks the corpus changes, while exit-code coverage and multi-turn are what the rebalance restored and added (training budget and dataset size moved too, so this is a pattern across three runs, not an isolated cause).

The held-out exit metric is asymmetric. The 25 wider codes in the 40-item exit score were in the baseline's training set but not the released model's. So the same 36/40 means different things: the released model *generalizes* to 23 of the 25 wider codes, while the baseline *recalls* 22 of 25 (it saw them in training). The only truly comparable subset is the legacy 15, where the baseline edges ahead 14 to 13; the released model makes that up on print (7/8 against 6/8), hence the 20/23 tie. Because the wider codes measure recall for one model and generalization for the other, the exit metric is never aggregated across models.

Baseline provenance. The baseline's published (merged) weights score 20/23; its unmerged final adapter scores 19/23. The single difference is `exit(249)`, a borderline `0xF9` conversion that the merge's fp32→bf16 cast happens to flip from fail to pass.[12] The released model shows no such gap.

Base model. The untuned base model scores 0% on the same harness.

Learning dynamics. Valid ELF structure shows up by roughly step 200, while operand binding (the right value in the right field) lags behind throughout. The model learns the scaffolding before the contents.

12. The whole 82.6%/87.0% distinction for the baseline hinges on this one item; the released model is unaffected, scoring 20/23 both as an adapter and merged.

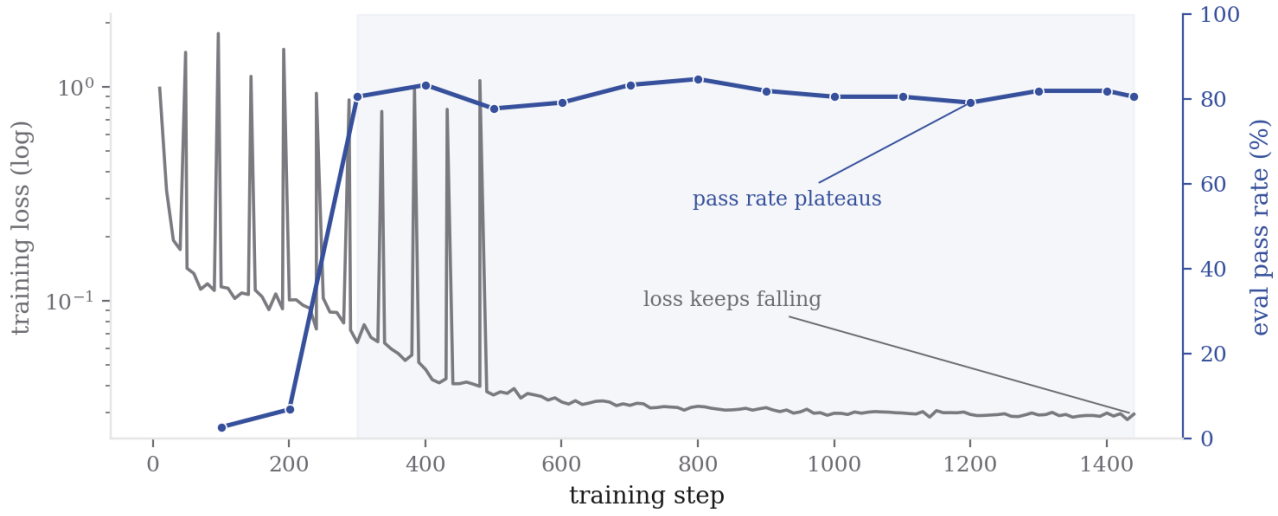


Figure 2 Loss–pass-rate decoupling on the released run. Training loss falls by nearly two orders of magnitude while the training-time evaluation pass rate (72-item set) climbs to a plateau near 80% by step 300 and thereafter only swings; the two curves move independently. The earlier packing run showed the same decoupling more starkly: loss collapsed to 0.04 by step 10 while the pass rate stayed near 20%.

8 Failure Analysis

The two models fail at different *stages*. The released model's failures are almost all behavioral (valid hex, valid ELF, wrong operand), while the baseline still produces the occasional hex or structure failure (Table 4). Structure is learned first and stays learned; what remains is operand binding.

Table 4 Failure-stage composition on a 47-item held-out subset (legacy 23, print-and-exit 8, novel strings 12, multi-turn 4). The released model carries no structural failures; the baseline still does.

Run	Hex	Structure	Behavior
v1.1 (`jul26` adapter)	0	0	11
v1 (`jun14` merged)	3	3	13

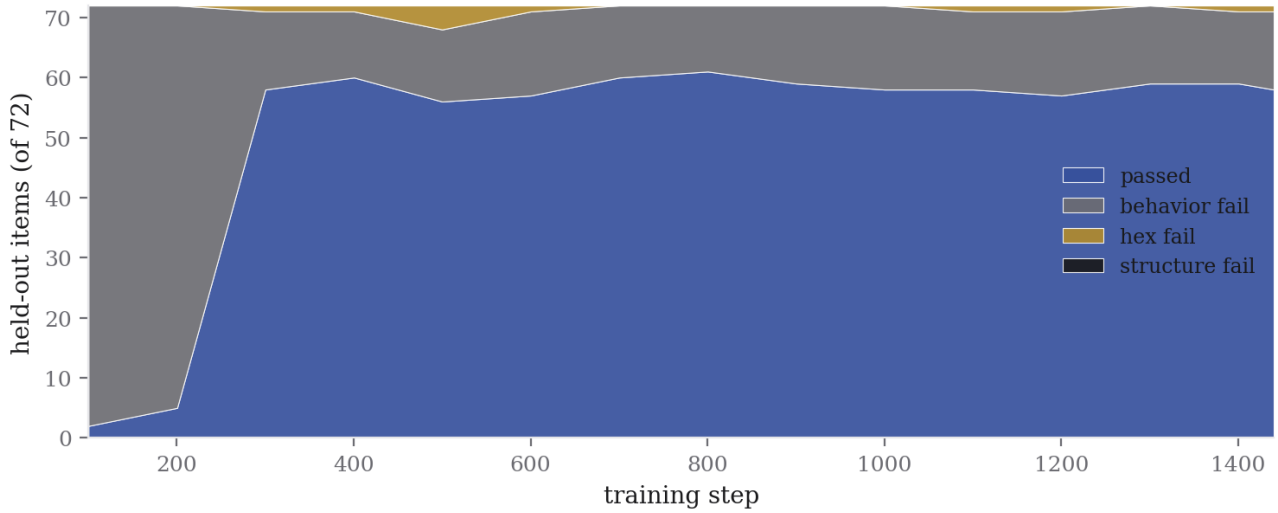


Figure 3 Training-time evaluation composition over the released run by harness stage (72-item set). Passing outputs rise sharply by step 300; behavior-stage failures make up nearly all of the remainder and clear as training proceeds, while hex-stage failures appear only as occasional late-run flips and structure-stage failures are absent, consistent with Table 4.

Converting arbitrary decimal values to hex and counting characters are still persistent weaknesses, for example the `write` length operand (`rdx`) against the actual payload length. Overcounts read past the payload: `yeet` is emitted with `rdx` 6 against a 5-byte `yeet\n` payload, so a trailing NUL gets read. Undercounts drop bytes: `boop` is emitted with `rdx` 4, dropping the newline. Short strings (`hi` , `bye`) are much more reliable.

Out-of-distribution inputs fail harder. In distribution, the ELF format is encoded correctly and only specific field values slip; far enough outside the training distribution, basic formatting and encoding fail too.

Novel multi-word strings can break as early as hex encoding (odd-length output) or the structure stage (a corrupt program header, for example), let alone behavior. On the frozen set of 40 novel out-of-distribution strings, the baseline passes 8 (20%), far below its in-distribution 87%. The released model passes 26 (65%), the largest gain in items from the corpus rebalance. Because the dataset is generated programmatically, the held-out split draws from the same `SAMPLE_STRINGS` generator, so it is held out but still in distribution.

Exit codes come down to one byte. Every pure-exit binary in the training data is 142 bytes and differs from every other in exactly one byte, at offset 132: the exit-code immediate. So the exit half of the task is a pure decimal-to-hex conversion worth roughly 2 of about 470 tokens per example. The rest of the binary is shared scaffolding, which is why exit failures are behavioral rather than structural. The misses bear this out: they are near-miss conversions in both models (`exit(203)` →202, i.e. `0xCB` → `0xCA` , and 240→224 for the released model; 223→222 and 169→175 for the baseline). Two codes, 203 and 223, keep failing across otherwise different runs and eras. That is weak longitudinal evidence that a handful of two-digit hex conversions are genuinely hard.

String errors are scrambles, not garbage. Behavior-stage string failures are drops, insertions, transpositions and truncations of the correct byte sequence, not wholesale garbage. Sorted by edit type, the released model's behavior failures are 5 scrambles, 1 truncation and 1 transposition. The baseline's are 5 scrambles, 1 truncation, 1 insertion and 2 *length-only* misses (`boop` → `boop\n\0` : spelled perfectly, write length off by one), a milder class the released model does not produce. A doubled adjacent character makes scrambles more likely but is not required for them. On the 12 matched pairs (same characters, only whether they sit next to each other changes) the baseline passes 2/24 and the released model 18/24: better, but not clean. So doubled characters are treated as an aggravating factor, not a standalone claim.

Multi-turn. Multi-turn failures first showed up in ad hoc testing. On the frozen set they turn out to be the sharpest behavioral difference between the models: the released model passes 10 of 12 multi-turn conversations, the baseline 0 of 12. The baseline's second turn collapses *structurally* into a header-only stub with no program header and an entry point past the end of the file, the same shape seen independently in June and August. The released model's second-turn misses are behavioral near-misses; it drops one `d` from "daddy", emitting "dady". The baseline drops that same `d` , so the improvement is the recovered structure, not the spelling.

Measurement integrity. One harness bug is worth reporting because it would have inflated exactly the generalization numbers above. The sandbox and host execution paths reported signal deaths differently: `bash` reports a signal-killed child as `128+N`, while `subprocess` returns `-N`. Under the Docker path, four of the wider exit codes (133, 136, 139 and 159, which are 128 plus SIGTRAP, SIGFPE, SIGSEGV and SIGSYS) could therefore score a *crash* as a passing exit. SIGSYS is exactly what this model's signature wrong-syscall-immediate failure raises. The fix makes termination a typed value (`Exited` versus `Signaled`) parsed at each boundary, so a crash and an exit with the same number can no longer compare equal. The frozen-set results in this report were measured after the fix, and earlier published numbers came from the host path, so neither is affected.

9 Discussion

An executable verifiable reward beats the usual proxy metrics. At every stage of training, loss or token accuracy decoupled from task success: 99.4% token accuracy on a dataset with a sampling gap (Section 4); a loss of 0.04 on packing-contaminated sequences (Section 5); a checkpoint that passed evaluation but failed in serving (Section 6). Execution is the honest signal. The sharpest case is the intermediate run (v1.1-alpha). At 0.038 loss, with every exit-code binary structurally valid, it generalized to essentially none of the held-out exit codes (1 of 40) and got only 1 of the 12 codes it had *trained on* right. A near-zero score on codes the model trained on rules out a quirk of the holdout: it fails to place the value even for codes it has seen. The two signals most tempting to trust, loss and structural validity, were both saturated while the model was failing the task (Figure 4).

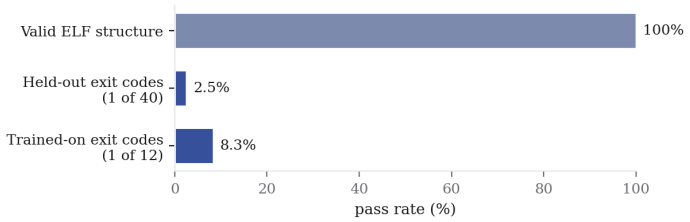


Figure 4 Proxy signals against task success for the intermediate run (v1.1-alpha). Training loss (0.038) and structural validity on exit-code binaries (100%) both look converged, yet the model generalizes to almost no held-out exit codes and fails even on codes it trained on. The trained-code control shows this is a capability floor, not a holdout artifact.

The reward is ready for RL. The shaped evaluation score is deterministic and cheap to compute. Supervised fine-tuning plateaus in the 80% range, which makes reinforcement learning on top the obvious next step: the evaluation harness can already serve as the reward function for

a GRPO/RLVR run [12, 4, 7]. This is future work.

Infrastructure and serving lessons. Packing needs boundary masking, and in our stack the failure without it was silent [11, 3]. A correct checkpoint still fails if the prompt template, sampling or any other serving setting drifts from training; byte-exact output has no margin for error. We think this end-to-end fragility is one of the more useful findings here.

Limitations. These results come from one base model, fine-tuned with LoRA only, decoded greedily and served at one precision (f16), with no reinforcement learning yet. Novel strings improved but are still the weakest axis: 65% (26/40) on strings outside the training distribution, against 87% (20/23) on the legacy held-out set.

The three campaign runs were not a controlled experiment. Corpus composition, the exit-code budget and dataset size all changed between runs, so no gain can be credited to any one of those changes. The runs are read as three points along one evolution, not as isolated causes, and with no further training runs planned, this report cannot separate their effects.

The test sets are small: 8 to 40 items per axis, including 12 matched string pairs. The counts describe how the model did on these items, not its accuracy on prompts in general or how much a rerun would vary.

On the legacy set and the held-out exit codes, v1.1 matches v1 (20/23 and 36/40 each). A paired McNemar test, which compares two models on the same items by looking only at the items where they disagree, finds no difference ($p \approx 1$). Finding no difference is not proof the models are equal, and it does not show these capabilities are independent of the corpus. The real differences are on the robustness axes, and with counts this small they are directions, not precise estimates.

10 Related Work

Code language models from Codex [1] onward emit *source* and are judged on the compiled or interpreted result; CRAP emits the *final bytes* without a compiler and checks them by running them. The evaluation-as-reward design follows RLVR [12], rule-based rewards at scale in DeepSeek-R1 [4] and execution-feedback fine-tuning with held-out pass/fail rewards in RLEF [7], which is the closest method to our planned GRPO direction. AlphaDev [14] is the nearest spiritual neighbor (learning at the assembly level with correctness tests as the signal), with AlphaCode [13] as its source-level companion. The deliberately minimal approach follows TinyStories [6] and phi-1 [9]. Parameter-efficient fine-tuning uses

LoRA [10]; QLoRA [5] is left as future work.

11 Conclusion

CRAP's contribution is methodological: a minimal, fully verifiable test bed for fine-tuning grounded in execution, plus four findings that should carry over to other projects. Loss and token accuracy decoupled from task success at every stage, most sharply in a run that failed even the exit codes it trained on. Robustness tracked corpus composition, though three runs that changed several things at once cannot separate it from training budget. Sequence packing without boundary masking silently contaminated training. And a correct checkpoint was undermined by the serving stack.

Every program ends up as a binary, so every other coding task is, strictly speaking, a downstream application of this one. We therefore consider coding solved, for programs that print a string and exit. That said, a compiler computes every offset deterministically, correctly and in milliseconds; CRAP approximates them, more slowly, with several gigabytes of weights and occasionally incorrectly. Traditional programming toolchains are safe… for now.

AI Disclosure

Language models were used throughout this project: as the subject of study, as a debugging and development aid for the training and serving infrastructure, and as a writing assistant used in a manner analogous to pair programming. The author, a software developer by background rather than an academic, used the tool iteratively throughout the writing process: drafting a section, reviewing and critiquing, revising based on that exchange and repeating. The intellectual content is the author's own work, developed and documented independently in an accompanying essay (blog.studiohaynes.com/go/crap). The tool's contribution was in the back-and-forth of shaping that content into academic-paper format: catching register mismatches, flagging structural gaps and suggesting revisions that the author then accepted, modified, or rejected. The final text reflects that iterative process, not a single generation. All experiments, measurements and mistakes are the human's fault. No language model emitted any part of this paper directly as a Linux binary, regrettably.

References

[1] Chen, M., et al. (2021). Evaluating Large Language Models Trained on Code. *arXiv:2107.03374*.

Introduces Codex and the HumanEval benchmark: generated code is judged by executing it against unit tests, not by text similarity. CRAP's parse–validate–execute harness is the same idea applied one level lower, to the executable itself.

[2] Dao, T., Fu, D., Ermon, S., Rudra, A., & Ré, C. (2022). FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *arXiv:2205.14135*.

IO-aware exact attention that tiles the computation to avoid materializing the attention matrix. Background for the variable-length attention kernels whose absence caused the packing contamination of Section 5.

[3] Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv:2307.08691*.

Faster successor with the `cu_seqlens` variable-length interface: the kernel CRAP needed for boundary-safe sequence packing, and could not build on the Blackwell + cp314 stack.

[4] DeepSeek-AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv:2501.12948*.

Large-scale RL against rule-based, automatically checkable rewards, with no learned reward model. Evidence that verifiable rewards of CRAP's kind scale far beyond toy settings.

[5] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. *arXiv:2305.14314*.

Trains LoRA adapters over a 4-bit quantized base model, drastically cutting finetuning memory. Relevant to the quantized-base variants left as future work.

[6] Eldan, R., & Li, Y. (2023). TinyStories: How Small Can Language Models Be and Still Speak Coherent English? *arXiv:2305.07759*.

Shows that tiny models trained on a deliberately narrow synthetic corpus exhibit coherent skill within that domain. Licenses CRAP's minimal-testbed posture: shrink the domain, not the rigor.

[7] Gehring, J., et al. (2024). RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning. *arXiv:2410.02089*.

RL fine-tuning of code models grounded in execution feedback from held-out tests. The closest methodological neighbor to CRAP's planned GRPO direction.

[8] Google DeepMind (2026). Gemma 4 model card: *google/gemma-4-E2B-it*. https://huggingface.co/google/gemma-4-E2B-it. An-

nouncement: https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/.

Model card for the small (E2B) instruction-tuned Gemma 4 variant, with native system-prompt support and optional thinking. The base model fine-tuned here, used text-only with thinking disabled.

[9] Gunasekar, S., et al. (2023). Textbooks Are All You Need. *arXiv:2306.11644*.

phi-1: a small model trained on curated "textbook-quality" data outperforms much larger ones on code. Companion evidence that data quality and task narrowing beat scale in the small regime.

[10] Hu, E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv:2106.09685*.

Freezes base weights and trains low-rank update matrices instead, cutting trainable parameters by orders of magnitude. The method behind the 24.2M-parameter adapter trained here.

[11] Krell, M., Kosec, M., Perez, S., & Fitzgibbon, A. (2021). Efficient Sequence Packing without Cross-contamination. *arXiv:2107.02027*.

Shows sequence packing is safe only with attention masking that respects document boundaries. The paper we should have read before the packing run (Section 5).

[12] Lambert, N., et al. (2024). Tülu 3: Pushing Frontiers in Open Language Model Post-Training. *arXiv:2411.15124*.

Names and systematizes RLVR: reinforcement learning against automatically verifiable rewards. CRAP's harness is a drop-in reward in exactly this sense.

[13] Li, Y., et al. (2022). Competition-Level Code Generation with AlphaCode. *arXiv:2203.07814*.

Competition-level code generation via massive sampling filtered by execution on example tests. Source-level companion to AlphaDev: execution as filter rather than reward.

[14] Mankowitz, D., et al. (2023). Faster sorting algorithms discovered using deep reinforcement learning. *Nature*, 618, 257–263.

AlphaDev discovers faster sorting routines by RL directly over assembly instructions, verified by correctness tests. The nearest spiritual neighbor: learning below source level with execution as the signal.

[15] Moadel, D. (2026, June 8). Elon Musk: Coding Was a Top Job for Decades. It Will Be Dead By the End of the Year. *Yahoo Finance*. https://finance.yahoo.com/sectors/technology/articles/elon-musk-coding-top-job-163850758.html.

Reports Musk's prediction that AI will soon skip source code and create binaries directly. The prompt for this project, which began as a joke about it.

A Hello-World Size Gradient Sources

Source for each rung of the size gradient in Section 1. Each program prints a short string and exits; sizes are the measured sizes of the built artifacts in the repository's `reference/` tree.

- main.rs (default Rust release build, 445,904 bytes)

```
fn main() {
    println!("Hello, World!");
}
```

- hello.c (default C build with clang, 16,056 bytes)

```
#include <stdio.h>

int main()
{
  printf("hello world\n");
  return 0;
}
```

- hello_by_hand.s (hand-written raw-syscall assembly, 8,848 bytes)

```
  .intel_syntax noprefix     # Use Intel syntax instead of
    AT&T syntax for much better readability
  .global _start             # The standard entry point for
    Linux ELF binaries

  .text                      # Switch to the executable code
     section
_start:                      # Label marking the precise
    entry point
  # --- SYSTEM CALL: sys_write (Print to screen) ---
```

```
  mov rax, 1                # Syscall Number: 1 (sys_write
    in Linux x86_64)
  mov rdi, 1                # Argument 1: File descriptor
    for Standard Output (stdout = 1)
  lea rsi, [rip + str]      # Argument 2: Load the address
    of our string into rsi (using instruction-pointer
    relative addressing)
  mov rdx, 14               # Argument 3: The exact length
    of our string in bytes (14 characters)
  syscall                   # Trap to the Linux kernel to
    execute

  # --- SYSTEM CALL: sys_exit (Terminate program cleanly) ---

  mov rax, 60               # Syscall Number: 60 (sys_exit
    in Linux x86_64)
  mov rdi, 0                # Argument 1: Exit status code
    (0 = success)
  syscall                   # Trap to the Linux kernel to
    instantly kill the process

  .data                     # Switch to the data section
str:                        # Label marking the memory
    location of our text
  .ascii "Hello, World!\n"  # The string itself as raw
    bytes
```

- hello_minimal (minimal print-and-exit ELF, 193 bytes)

```
# ELF header, 64 bytes (0x00-0x3f); e_entry = 0x400078
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0200 3e00 0100 0000 7800 4000 0000 0000  ..>.....x.@.....
00000020: 4000 0000 0000 0000 0000 0000 0000 0000  @...............
00000030: 0000 0000 4000 3800 0100 4000 0000 0000  ....@.8...@.....
# One PT_LOAD program header, 56 bytes (0x40-0x77);
# p_filesz = p_memsz = 0xc1 = 193 = the file length
00000040: 0100 0000 0500 0000 0000 0000 0000 0000  ................
00000050: 0000 4000 0000 0000 0000 4000 0000 0000  ..@.......@.....
00000060: c100 0000 0000 0000 c100 0000 0000 0000  ................
# Code from 0x78 (after p_align at 0x70-0x77):
# write(1, str, 12); exit(0); lea rsi, [rip+0x22] -> str at 0xb5
00000070: 0000 2000 0000 0000 48b8 0100 0000 0000  .. .....H.......
00000080: 0000 48bf 0100 0000 0000 0000 488d 3522  ..H.........H.5"
00000090: 0000 0048 ba0c 0000 0000 0000 000f 0548  ...H...........H
000000a0: b83c 0000 0000 0000 0048 bf00 0000 0000  .<.......H......
```

```
# Payload at 0xb5: "hello world\n" (12 bytes; rdx = 0x0c)
000000b0: 0000 000f 0568 656c 6c6f 2077 6f72 6c64  .....hello world
000000c0: 0a                                       .
```

B Baseline Training Progression

The baseline (v1, `jun14`) scored on the legacy 23 every 50 steps during training. These are training-time scores of the *unmerged* adapter at each checkpoint; the published merged weights score 20/23 (Section 7). Loss falls smoothly throughout while the pass rate climbs to its 87.0% peak by step 300 and then swings within a few binaries of it, the loss–pass-rate decoupling of Figure 2 in miniature. The saved adapter is the final step 620, not a peak (Section 5).

Table 5 Baseline per-step progression on the legacy split. Training loss is the logged value at the step; the final step logged no separate loss.

Step	Train loss	Pass rate	Passed
50	0.064	8.7%	2/23
100	0.048	17.4%	4/23
150	0.042	17.4%	4/23
200	0.037	43.5%	10/23
250	0.035	73.9%	17/23
300	0.033	**87.0%**	20/23
350	0.032	82.6%	19/23
400	0.027	73.9%	17/23
450	0.027	87.0%	20/23
500	0.027	87.0%	20/23
550	0.025	87.0%	20/23
600	0.025	82.6%	19/23
620	n/a	82.6%	19/23

C Reproducibility

Every artifact needed to reproduce the results already exists; this appendix collects the pointers.

Evaluation set. Frozen and versioned at `EVAL_SET_VERSION = 2`, 144 entries, content fingerprint `89da56f51af9abdd…`. A test recomputes the fingerprint from the held-out literals, so any silent change to a list fails. The legacy split is a random holdout under fixed seed 99.

Released-model corpus. 3,834 examples, seed 42, three repeats, Linux-only; SHA-256 `4eb6723b17edc607…` . Held out of training: 15 legacy exit codes and 25 wider exit codes, while the 12 trained-code control codes were deliberately *kept in*. Full digests ship in the model and dataset metadata.

Adapters. Each run's saved adapter is identified by the SHA-256 of its `adapter_model.safetensors` (abbreviated): baseline / v1 (`jun14` , checkpoint-620) `f9fcca96fdb8f44a` ; v1.1-alpha (`jul19` , checkpoint-450) `6e278db8…` ; released / v1.1 (`jul26` , checkpoint-800) `e11ae64c…` .

Weights. Under the `matthewhaynesonline` organization, the released model is `gemma-4-E2B-it-crap-1.1` (with `…-1.1-gguf` for the GGUF quantizations); the baseline remains at the unversioned `gemma-4-E2B-it-crap` . Code, the binary-gen library and the evaluation harness are in the project repository linked in the masthead.